

Software Requirement Specifications
Web Vulnerability Scanning and Testing



Submitted by

Hannan Fiaz

Submitted to

Musarrat Kareem

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Summary

This document presents Secure Deep Scan, a free and open-source web vulnerability scanner built to solve the problem of costly and complex security tools. Research shows that while OWASP Top 10 vulnerabilities like SQLi and XSS are common, most existing scanners need expensive monthly subscriptions or technical setup, which prevents students and small businesses from testing their websites. Our findings confirm that a simple no-login system can use OWASP standards to scan a URL, detect common flaws, and generate a PDF report. The project requires only a single-user interface, a two-table database for URLs and reports, and an automated scanning engine, making web security testing accessible to everyone without financial or technical barriers.

Contents

1. Introduction	4
1.1. Purpose.....	4
1.2. Scope	4
1.3. Product Perspective	5
1.4. User Characteristics.....	6
1.5. Similar apps and systems/Literature Review	6
1.6. Proposed Technologies	8
2. Requirements	8
2.1. Function Requirements	9
2.1.1. User Input	9
2.1.2. Automated Web Crawling	9
2.1.3. Vulnerability Detection.....	9
2.1.4. Real Time Progress Tracker	10
2.2. Non-Functional Requirements	10
3. Use Cases and Flow of Processes	11
3.1.....	11
Use Case 1	12
Use Case 2	13
Use Case 3	14
3.2 Sequence Daigram	14
3.3 Data Flow Diagram.....	15
3.2 ER Diagram	16
3.4.1	17
4 References	18

1. Introduction

In today's digital era, an online presence is a business necessity. However, this online presence introduces significant security risks. Despite standard defenses, hidden vulnerabilities (such as backdoors and logic loopholes) remain frequent targets for malicious-actors/hackers seeking to steal confidential data.

This project, Secure Deep Scan, is developed to address this critical gap. By implementing an automated Web Vulnerability Scanning and Testing system, this tool provides businesses with a systematic method to audit their digital assets. It focuses on identifying risks like those defined in the OWASP Top 10, ensuring that security is not an afterthought, but a core component of the business's online infrastructure and handle accordingly.

1.1. Purpose

The purpose of this project is to provide a special environment where businesses can carefully audit their web applications for security flaws (like SQLi, XSS, CSRF etc). Unlike system integrated plugins, this web system operates as an external auditor, simulating the perspective of an attacker to identify vulnerabilities before they can be exploited vulnerability. It aims to bridge the gap between "being online" and "being secure" by offering a centralized platform for security assessment.

1.2. Scope

The scope of this project is to design and implement an automated **Dynamic Application Security Testing (DAST)** tool that bridges the gap between complex security software and the needs of small businesses. The project focuses on the following core functionalities and boundaries:

- 1) **Basic User Interface:** A simplified graphical user interface (GUI) or web dashboard will be developed to allow non-technical users to initiate scans and monitor progress.
- 2) **Automated Discovery & Crawling:** System will use Python libraries (Like BeautifulSoup 4 API) to perform automated link discovering. This help in navigate the target URL to identify all accessible pages, sub pages, and input entry points such as search bars, login forms, etc.
- 3) **Vulnerability Assessment Engine:** By using **OWASP ZAP API**, system will conduct active scanning against the most common and known attacks. This assessment is specifically to detect

the most known **OWASP Top 10** threats, including SQL Injection (SQLi), Cross-Site Scripting (XSS), and missing security headers (e.g., HSTS, X-Frame-Options).

- 4) **Simplified Reporting:** The system will feature an automated reporting module that transforms raw JSON or XML scan data into a human-readable pdf format. These reports will categorize on based of severity (High, Medium, Low) and provide plain-English remediation steps to help business owners fix identified flaws.
- 5) **Excluded Operations:** following are considered out of the scope:
 - a) **Static Analysis:** This web will not review the application's source code.
 - b) **Post-Exploitation:** The web system will identify vulnerabilities but will not attempt to exploit them or try to gain unauthorized access to the server.
 - c) **Network/Infrastructure Testing:** The scope of web is limited to the web application layer and does not include testing for server-level OS vulnerabilities or network firewall strength.

1.3. Product Perspective

This project is an independent, standalone Dynamic Application Security Testing (DAST) tool designed to work as a self-contained security assessment website. And its functions provide a primary security auditor for small business web applications, its perspective within the broader software ecosystem is defined by the following relationships:

- **Relationship with Existing Security Engines:**

The project does not aim to re-invent core scanning logic from scratch. Instead, it serves as an abstract integration layer that wraps the professional-grade OWASP ZAP API. It acts as a bridge, translating the complex, high-volume data from OWASP ZAP into a simplified format suitable for non-security experts.
- **System Environment:**

The application is a Python-based software website. It interacts externally with website applications of user by simulating HTTPS requests and analyzing server responses of user. It can work with websites without knowing of underlying technology, meaning it can scan websites built in PHP, Java, JavaScript, or any other web technology without needing access to the underlying server hardware or source code.

- User Perspective (Target Audience):**
 From the perspective of a small business owner, this tool is an automated consultant. Unlike enterprises tools like **Burp Suite Professional**, which require in-detail cybersecurity knowledge, this project is designed as a "single-click" solution that provides user ease and clear reporting over complex manual configuration.
- Integration with Development Lifecycle:**
 This website is used before deploying the production server to validate user system completely. It is intended to be used at the end of the development phase to catch "low-hanging fruit" vulnerabilities before the site goes live, serving as a first line of defense in a secure software development lifecycle (SSDLC).

1.4. User Characteristics

The users of this project are categorized into three groups based on their technical knowledge and needs:

- Small Business owner
 - a) Technical Skills: don't know much about cybersecurity
 - b) System Interaction: single button to click and generate plain text report.
- Junior Web developer
 - a) Technical Skills: Medium knowledge
 - b) System Interaction: single button to click and look for specific URL details where code has vulnerability and so, they can fix code issues.
- Expert
 - a) Technical Skills: Expert level field knowledge
 - b) System Interaction: single button to create report. To test system functionality. And to automate their repetitive task.

1.5. Similar apps and systems/Literature Review

This field is also filled by a variety of different tools and systems ranging from professional enterprise platforms to lightweight open-source scripts/systems. For a small business, the choice often falls between high-cost automation and complex manual tools. This section reviews existing systems that influenced the development of this project.

Given below Table gives more explanation of the similar product available in markets and also their comparison with my project.

Table 1.5.1: Comparative Analysis of Existing Systems with my project

Feature	Commercial Tools (e.g., Burp Suite Pro, Acunetix)	Open-Source Tools (e.g., OWASP ZAP, Nikto)	My Project
Target	Enterprises and Cyber Experts	Security Researchers and Developers	Small Businesses / Non-Technical Users
Cost	Expensive. Need to buy License	Free / Open-Source	Free (Open-Source integration) built by me.
Ease of Use	complex	Requires technical configuration	Simplified "One-Click" Interface
Reporting Format	Technical / Forensic	Detailed / Raw Data	Plain English / Actionable Advice
Hardware Needs	High resource consumption	Moderate	Lightweight (Python-based)
Setup Time	Lengthy (Registration/Install)	Moderate (Config required)	Rapid (Pre-configured modules)

1.6. Proposed Technologies

The project will be built on following tech stack. Reason for selecting them is given below: -

1.6.1. Programming Language

Python: Industry standard for automation in cybersecurity. Offers extensible libraries to work.

1.6.2. Security Engine

OWASP ZAP: Fully open source , provides powerful API to work with python.

1.6.3. Web Scarping

Beautiful Soup 4: ensure all possible attack surfaces found. Parse the HTML file to find all <a href> and <Form> tags.

1.6.4 UI

Frontend: HTML , CSS are used to build basic dashboard to interact with user.

1.6.5 Database

MySQL: Easy to use and perfect for small business.

1.6.7 Reporting Module

Jinja 2 & ReportLab: These 2 are used to create professional level pdf for reports. Jinja 2 is used to extract data from database/code functions and reportLab is used to create a PDF version of data in plain text format.

2. Requirements

First, you'll just enter a website address into a simple box. That starts an automatic process using a tool called Beautiful Soup 4, which explores the site. It basically maps out how the whole website

is put together, finding all the pages and forms it can access. Once we understand the site's layout, the real work begins with our scanning tool, OWASP ZAP. It actively checks for security problems by sending various test data to the site. This helps it find serious flaws like SQL injection, cross-site scripting (XSS), or if important security settings are missing. While it's doing this, the system constantly watches what the server sends back to spot any potential threats, always making sure it only scans the areas you've specified. After the scan is done, all the raw technical information gets collected and turned into a thorough report. This report categorizes all the issues based on how serious they are. The final output gives you clear, easy-to-understand explanations of the risks and threats, along with practical steps you can take to fix them. This way, you can understand and address your security weaknesses yourself, without needing an expert.

2.1. Function Requirements

Authors will provide definite number of functional requirements in standard format. These requirements are directly in the with functions which are already provided. The requirement format is as follows.

2.1.1. User Input

Name: FR001

Purpose: To lock the target and work on it.

User(s): small business owner, junior developer, Experts

Input:

URL: enter the valid URL of website

Depth: Enter the Depth of scanning

Output: System start working on URL to find vulnerabilities.

2.1.2. Automated Web Crawling

Name: FR002

Purpose: To discover all links and input fields like form, search bar etc.

User: System

How: Using beautiful soup 4 to discover all links and fields.

Output: Find out the System links and input fields and provide to OWASP engine.

2.1.3. Vulnerability Detection

Name: FR003

Purpose: To perform scanning and testing.

User(s): System

How : Using OWASP ZAP API system perform this activity.

Output: Find vulnerability in scanning and provide alerts, solutions and risks.

2.1.4. Real Time Progress Tracker

Name: FR004

Purpose: To tell user about progress of scanning/validating.

User(s): System.

How: Using OWASP ZAP API status function to performs this activity.

Output: Text Based progress of processes is shown.

2.1.5. Report Generation

Name: FR005

Purpose: To provide descriptive details of vulnerabilities and their solutions.

User(s): System.

How: By using ReportLab to perform this activity.

Output: PDF Based descriptive result will be generated.

2.2. Non-Functional Requirements

Non-functional requirements describe the quality attribute of system essentially how the system should perform rather than what it should do. Here is list of non-functional requirements of our system.

- **Usability:** The interface must be intuitive enough for a non-technical small business owner to operate with minimal training. Technical jargon must replace with plain-English explanations in all user-facing reports.
- **Performance & Efficiency:** The scanning process should be optimized to minimize the load on the target server. The crawler must implement timeouts to prevent the application from hanging on broken links or large files.

- **Accuracy:** The system should aim to minimize "false positives" by using the OWASP ZAP validation engine, ensuring that business owners don't waste time trying to fix non-existent security flaws.
- **Portability:** Since the tool is built with Python, it should be capable of running on various operating systems (Windows, macOS, Linux) without requiring significant changes to the source code.
- **Safety & Ethics:** The tool must be designed to perform "non-destructive" testing. It should identify vulnerabilities without causing system crashes, data loss, or denial-of-service (DoS) conditions on the client's website.
- **Reliability:** The system must handle unexpected errors (Like the target website going offline during a scan, Block from scanning) gracefully by notifying the user rather than crashing python code.

3. Use Cases and Flow of Processes

3.1

A use case is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved. The above figure shows the use cases of the user in the system. User is no needed to logged in. User can visit any time to perform use cases activities.

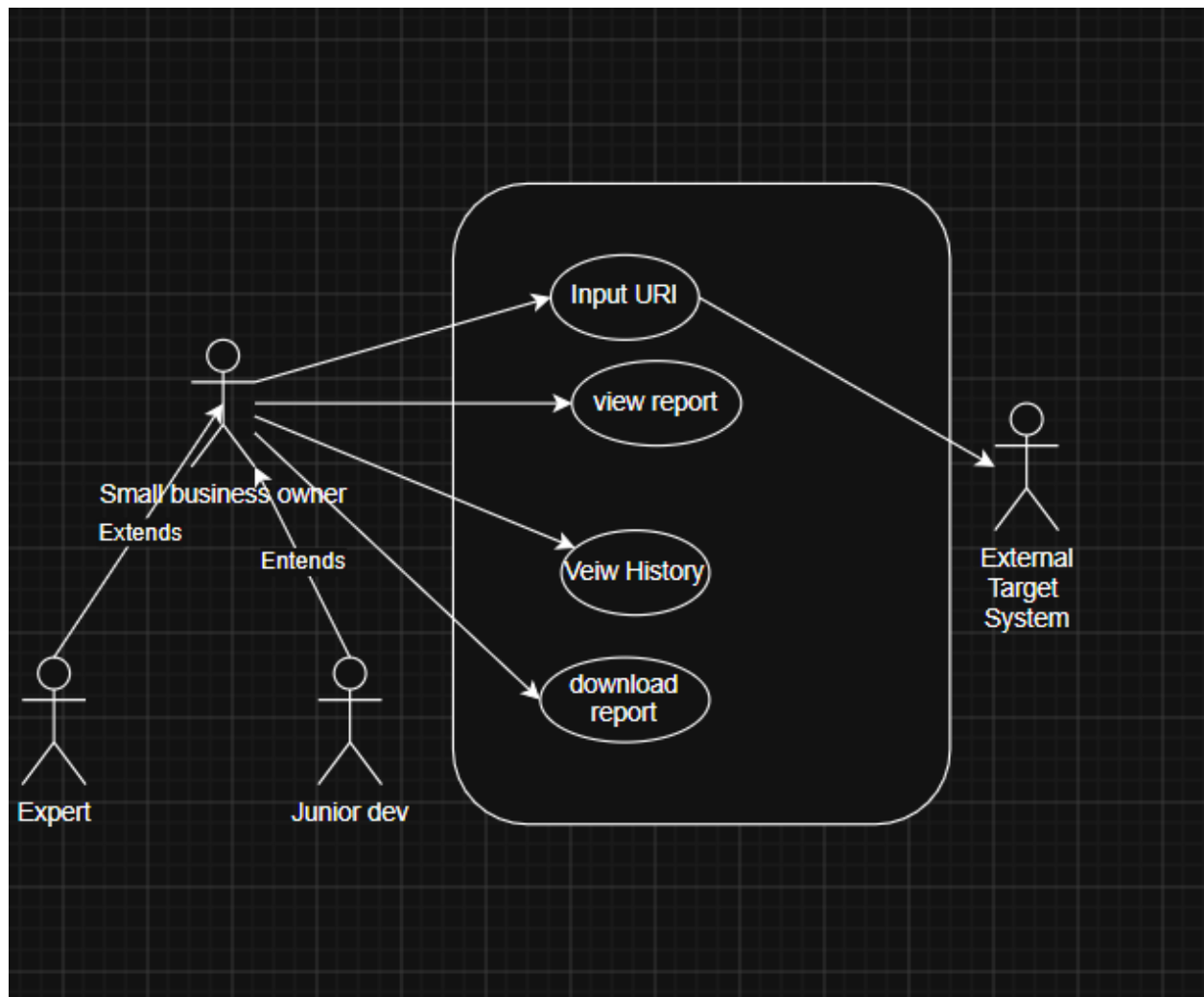


Figure 1: System Level Use Case Diagram

More detail about each use case is given below.

Use Case 1

ID	UC001
Name	Input URL
Description	In this use case, describes the process of user entering URL of target website

Requirement (s)	FR001
Actor(s)	Business Owner
Precondition	Target Website is ON/Available on internet when entering URL.
Workflow	<i>Basic Flow</i> 1. Actor enters URL 2. System validates the input URL 3. Valid URL forward for further process <i>Exceptions</i> 1. Invalid URL 2. System unable to response

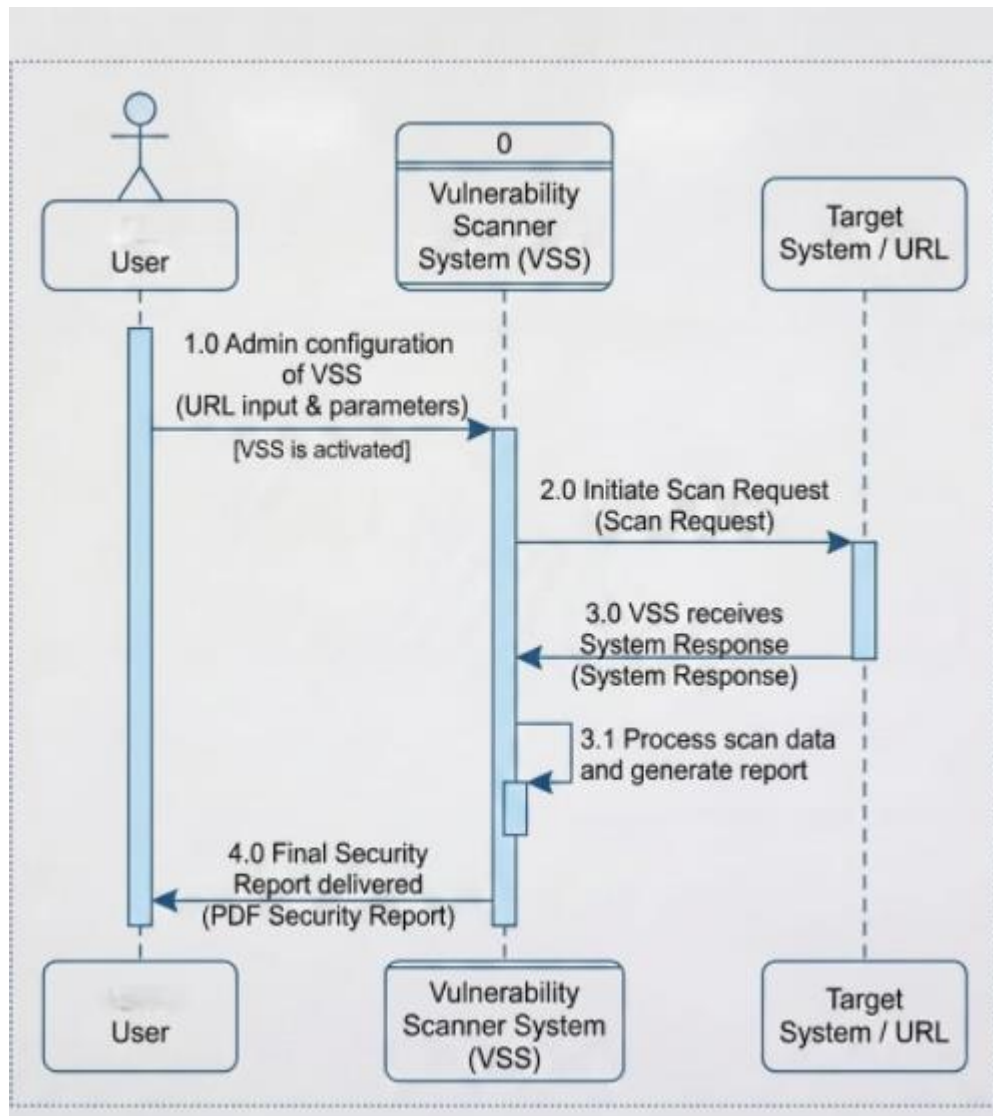
Use Case 2

ID	UC002
Name	View Report
Description	In this use case , describes the process of generating a new web page where users can view report.
Requirement (s)	FR001-FR004
Actor(s)	Business Owner
Precondition	FR001,002,003 and 004 complete without any interruption.
Workflow	<i>Basic Flow</i> 1. F001 to F004 complete 2. Front end page gathers necessary information from scanning engine. 3. After successfully loading data from backend, it will be presented to user. <i>Exceptions</i> 1. Any error in above FR(1,2,3,4). 1. System unable to response

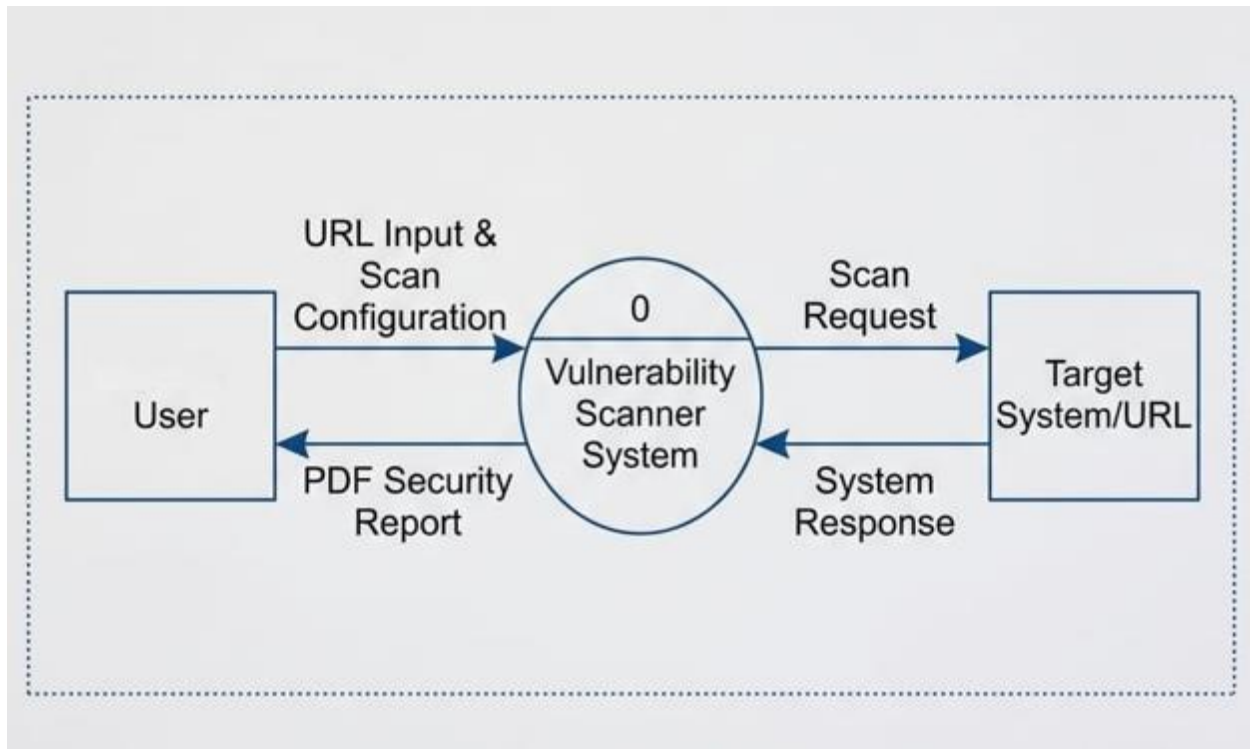
ID	UC003
Name	Download Report
Description	This use case describes the process of creating a new PDF report which users can download on their storage device.
Requirement (s)	FR005
Actor(s)	Business Owner
Precondition	FR001,002,003 and 004 complete without any interruption.
Workflow	<i>Basic Flow</i> 4. User click download button 5. New PDF file is created 6. All info gathers from engine and pastes in PDF file. 7. PDF file started downloading 8. A file with name of report eventually download in user storage device. <i>Exceptions</i> 2. While downloading network error. 3. System unable to response.

3.2 Sequence Daigram

Sequence Diagram is used to describe what is sequence of system and how sequentially users interact with it. It provides detail interaction of user with system from start to end step by step one after one in a sequence manner.



3.3 Data Flow Diagram



A data flow diagram (DFD) is a way of representing the flow of data of a process or a system. DFD describes the processes that engage in a system to transfer data from the input to the file storage and reports generation. The above figure shows the data flow of various processes in system.

3.2 ER Diagram

ER diagram mean Entity Relation diagram. As it stated earlier that we use relational database so we need ER diagram to explain how one table in database interact with other table. In this diagram (figure 000), An URL has many reports. And Each report connected only single URL.

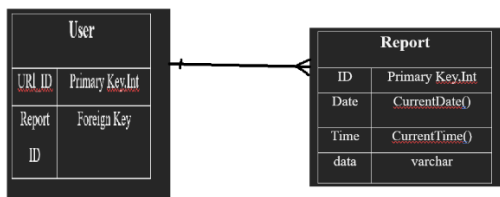
URL Table have only two attributes

1. URL_ID which is actual Url of target website
2. Report_ID which is foreign key. Connecting to report table.

3.4.1

Report Table has multiple attributes which are given below as: -

1. Report_ID: store unique Id of each report generated by system.
2. Time: store current time of report generation using system function.
3. Date: store current date of report generation using system function.
4. Data: store actual detailed report data which is provided by engine.



Assumptions and Dependencies

Users:

We have assumed that all the users using this platform are capable of connecting their devices to the Internet and navigate the browser on their devices to the address of secureDeepScan website.

Provider:

We have assumed that the Web will be running on a properly working web server and database system with an Internet connection that allows the system to perform all communications between the users.

The success of the system depends on:

- The performance of the platform is good.
- The information stored are secured and verified.
- The system is built with the perspective of HCI.

4 References

- [OWASP Foundation, “OWASP Top 10:2021,” 2021](#)
- [OWASP Foundation, “OWASP Web Security Testing Guide v4.2,” 2020](#)
- [K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical Guide to Information Security Testing and Assessment,” NIST Special Publication 800-115, Gaithersburg, MD, USA, 2008.\[3\] MITRE Corporation, “Common Weakness Enumeration,” 2026](#)
- [D. Stuttard and M. Pinto, The Web Application Hacker’s Handbook: Finding and Exploiting Security Flaws, 2nd ed. Indianapolis, IN, USA: Wiley, 2011.\[5\] FIRST.Org, Inc., “Common Vulnerability Scoring System version 3.1: Specification Document,” 2019](#)
- [L. Richardson, “Beautiful Soup Documentation,” 2024](#)